

Mini-projet

Intégration de prétraitement dans une Vidéo-capsule : cas de l'indice colorimétrique pour l'évaluation de la qualité de l'examen

Remarque :

J'utilise la carte [DE10 Nano](#) et non la DE0.

Etude préliminaire

Q1. Le coût d'une coloscopie est d'environ 765 euros tandis que le coût d'un examen par vidéocapsule est de l'ordre de 500 euros. L'utilisation d'une capsule endoscopique par rapport à un examen de vidéo-coloscopie fait économiser 256 euros.

Q2. La solution de la capsule endoscopique est moins chère et moins invasive et ne nécessite pas d'anesthésie.

Q3. Durée du transit de la vidéo-capsule :

$$T_{oesophage} = \frac{25 \text{ cm}}{3 \text{ cm/s}} = 8.33 \text{ s}$$

$$T_{estomac} = 4h$$

$$T_{intestin \text{ grêle} + gros \text{ intestin}} = \frac{8 \text{ m}}{2 \text{ cm/s}} = 400 \text{ min} = 6h40$$

$$T_{transit} = T_{oesophage} + T_{estomac} + T_{intestin \text{ grêle} + gros \text{ intestin}} = 10 \text{ h } 40 \text{ min } 8 \text{ secondes}$$

Q4. Nombre d'images prises par la vidéo-capsule :

$$N_{oesophage} = 8.33 \text{ s} \times 35 \text{ images/s} = 292 \text{ images}$$

$$N_{estomac} = 4h \times 2 \text{ images/s} = 28\,800 \text{ images}$$

$$N_{intestin \text{ grêle} + gros \text{ intestin}} = 6h40 \times 15 \text{ images/s} = 360\,000 \text{ images}$$

$$N_{tot} = N_{oesophage} + N_{estomac} + N_{intestin \text{ grêle} + gros \text{ intestin}} = 389\,092 \text{ images}$$

Q5. Taille de la mémoire nécessaire pour 389 092 images 320x280

$$M = 3 \times 320 \times 280 \times 389\,092 = 104\,587\,929\,600 \text{ octets} = 105 \text{ Go}$$

Q6. L'autonomie de la vidéo-capsule étant de 8h, on aura pas la fin du gros intestin mais on aura quand même une capture de l'ensemble du système digestif.

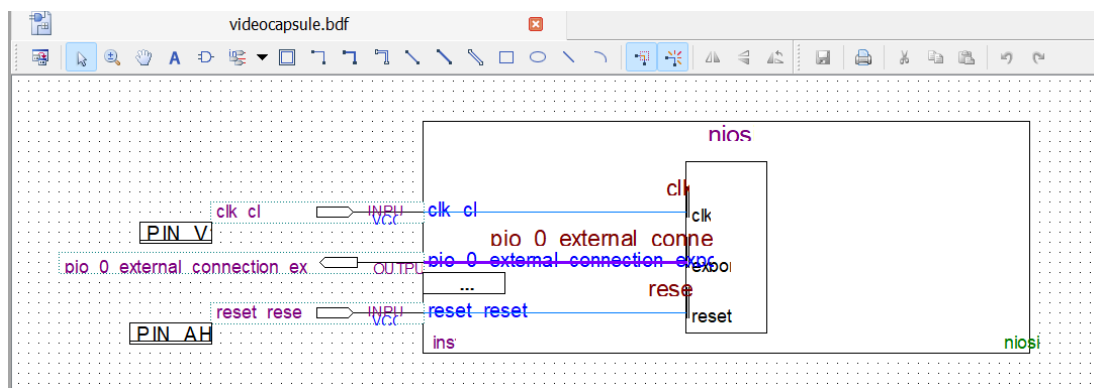
Partie 1 : Conception du processeur embarqué (Softcore) et programmation logicielle

Dans cette partie, nous allons concevoir un système SoPC avec un Nios2, en suivant le [Nios II Hardware Development Tutorial](#).

On obtient alors le système .qsys suivant :

Use	Connections	Name	Description	Export	Clock	Base	End	I...	Tags
✓		clk_0	Clock Source						
		clk_in	Clock Input	clk	exported				
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to Double-click to	clk_0				
		clk_reset	Reset Output	Double-click to Double-click to					
✓		onchip_mem...	On-Chip Memory (RAM o...						
		clk1	Clock Input	Double-click to Double-click to	clk_0 [clk1]	# 0x0000 8000	0x0000_cfff		
		s1	Avalon Memory Mapped ...	Double-click to Double-click to					
		reset1	Reset Input	Double-click to Double-click to					
✓		nios2_gen2...	Nios II Processor						
		clk	Clock Input	Double-click to Double-click to	clk_0 [clk]				
		reset	Reset Input	Double-click to Double-click to	[clk]				
		data_master	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]				
		instruction_m...	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]				
		irq	Interrupt Receiver	Double-click to Double-click to	[clk]				
		debug_reset_r...	Reset Output	Double-click to Double-click to	[clk]				
		debug_mem_...	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]	# 0x0001 0800	0x0001_0fff		
		custom_instru...	Custom Instruction Master	Double-click to Double-click to					
✓		jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to Double-click to	clk_0 [clk]				
		reset	Reset Input	Double-click to Double-click to	[clk]				
		avalon_jtag_sl...	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]	# 0x0001 1030	0x0001_1037		
		irq	Interrupt Sender	Double-click to Double-click to	[clk]				
✓		timer_0	Interval Timer						
		clk	Clock Input	Double-click to Double-click to	clk_0 [clk]				
		reset	Reset Input	Double-click to Double-click to	[clk]				
		s1	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]	# 0x0001 1000	0x0001_101f		
		irq	Interrupt Sender	Double-click to Double-click to	[clk]				
✓		sysid_qsys_0	System ID Peripheral						
		clk	Clock Input	Double-click to Double-click to	clk_0 [clk]				
		reset	Reset Input	Double-click to Double-click to	[clk]				
		control_slave	Avalon Memory Mapped ...	Double-click to Double-click to	[clk]	# 0x0001 1038	0x0001_103f		
✓		prio_0	PIO (Parallel I/O)						
		clk	Clock Input	Double-click to Double-click to	clk_0 [clk]				
		reset	Reset Input	Double-click to Double-click to	[clk]	# 0x0001 1020	0x0001_102f		
		s1	Avalon Memory Mapped ...	Double-click to Double-click to					
		external_conn...	Conduit	Double-click to Double-click to	prio_0_external...				

Une fois le code HDL du spoc généré, on peut également générer son symbole qu'on ajoute dans un fichier schéma bloc diagram .bdf. On connecte le sopc aux entrées nécessaires comme ci-dessous.



Une fois les entrées connectées, on les assigne au PIN correspondant dans le pin planner comme ci-dessous :

Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate
altera_...ved_tck	Input				PIN_AB5	2.5 V ...fault		12mA ...ault	
altera_...ved_tdi	Input				PIN_W10	3.3-V LVTTTL		16mA ...ault	
altera_...ved_tdo	Output				PIN_Y9	2.5 V ...fault		12mA ...ault	1 (default)
altera_...ved_tms	Input				PIN_AC7	2.5 V ...fault		12mA ...ault	
clk_clk	Input	PIN_V11	3B	B3B_N0	PIN_V11	3.3-V LVTTTL		16mA ...ault	
pio_0_e...port[7]	Output	PIN_W15	5A	B5A_N0	PIN_W15	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[6]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[5]	Output	PIN_V16	5A	B5A_N0	PIN_V16	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[4]	Output	PIN_V15	5A	B5A_N0	PIN_V15	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[3]	Output	PIN_AF26	5A	B5A_N0	PIN_AF26	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[2]	Output	PIN_AE26	5A	B5A_N0	PIN_AE26	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[1]	Output	PIN_Y16	5A	B5A_N0	PIN_Y16	3.3-V LVTTTL		16mA ...ault	1 (default)
pio_0_e...port[0]	Output	PIN_AA23	5A	B5A_N0	PIN_AA23	3.3-V LVTTTL		16mA ...ault	1 (default)
reset_reset_n	Input	PIN_AH17	4A	B4A_N0	PIN_AH17	3.3-V LVTTTL		16mA ...ault	
<new node>>									

Une fois que c'est compilé et synthétisé, on programme la carte avec l'outil programmer. On peut ensuite accéder grâce à l'outil Nios II Software Build Tools for Eclipse, avec laquelle on crée la BSP et une application qu'on pourra programmer et téléverser dans le Nios2. Pour nous familiariser avec la carte, on écrit alors le programme suivant, consistant à faire un chenillard avec les 8 leds de la carte :

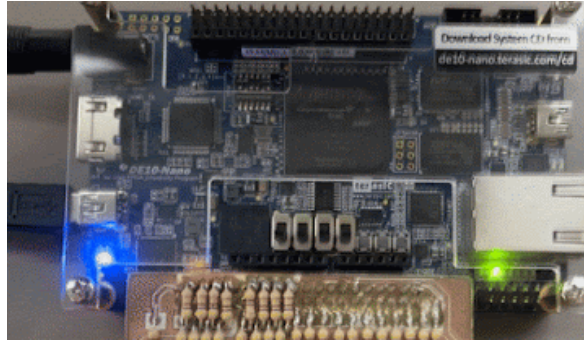
```
#include "altera_avalon_pio_regs.h"
#include "system.h"
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Hello from Nios II!\n");
    int led_value = 0x01;

    while(1)
    {
        IOWR_ALTERA_AVALON_PIO_DATA(PIO_0_BASE, led_value);
        led_value = led_value << 1;
        if (led_value == 0x100) led_value = 1;
        usleep(1000000);
    }
    return 0;
}
```

- La fonction macro `IOWR_ALTERA_AVALON_PIO_DATA()` est générée par la BSP et est disponible dans le fichier `altera_avalon_pio_regs.h`
- La macro `PIO_0_BASE` elle est disponible dans le fichier `system.h`

On obtient le résultat suivant :



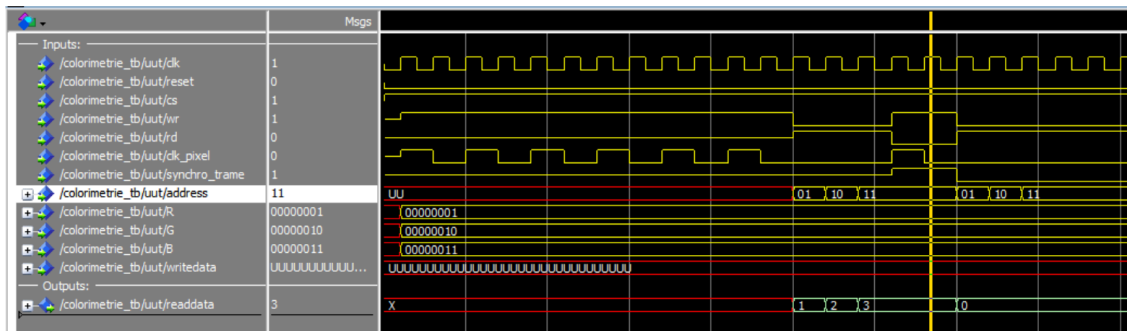
Partie 2 : IP indice colorimétrique

Dans cette partie, on a pour objectif de réaliser d'un IP en vhdl qu'on intégrera à notre SoPC. Cette IP qu'on nommera colorimetrie permet de calculer l'indice colorimétrique des images capturées, seules les images avec un indice suffisant seront traitées.

Cette IP devra être connectée au bus Avalon du softcore Nios II et comportera ainsi les signaux : address, writedata, readdata, write, read, chipselect. Mais aussi, bien évidemment un signal de reset et un clock système. En plus de ces signaux, on y ajoute les entrées R, G, B, clk_pixel et synchro_frame.

L'IP colorimétrie est conçue avec le programme vhdl [colorimetrie.vhd](#) en annexe.

Il est ensuite simulé pour être sûr qu'il suit le fonctionnement voulu, comme dans le chronogramme suivant :



Le chronogramme que vous mentionnez illustre le fonctionnement temporel de l'IP colorimétrie :

- Réinitialisation de Trame
Lorsque synchro_frame passe à '1' (sur un front montant de clk_pixel), toutes les sommes (R, G, B) et les compteurs de pixels sont remis à zéro.
- Accumulation des Pixels

Pour chaque pixel valide (sur le front montant de clk_pixel, si cs='1', wr='1' et synchro_frame='0'), les valeurs R, G, B sont additionnées aux sommes cumulées (sum_r, sum_g, sum_b) et le compteur de pixels (nb_value_r, etc.) est incrémenté.

- Lecture de la Moyenne

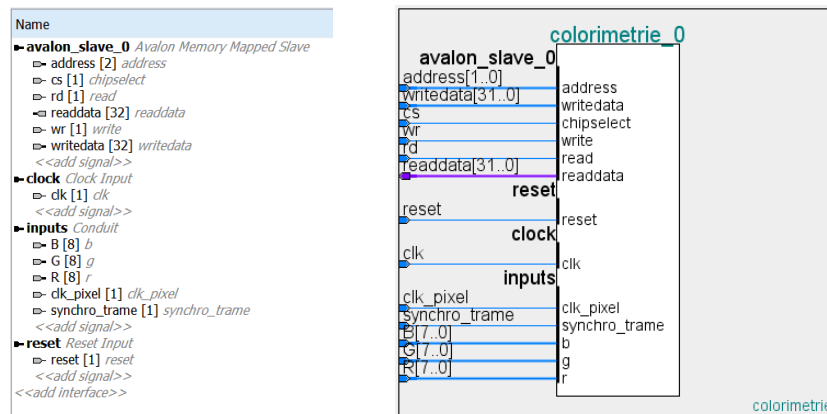
Lorsque le Nios II souhaite lire la moyenne (généralement après la fin de l'accumulation d'une trame) :

cs et rd passent à '1' (synchronisé sur clk).

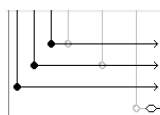
address est réglé sur '01', '10', ou '11'.

place le résultat (sum / nb_value) sur la sortie readdata.

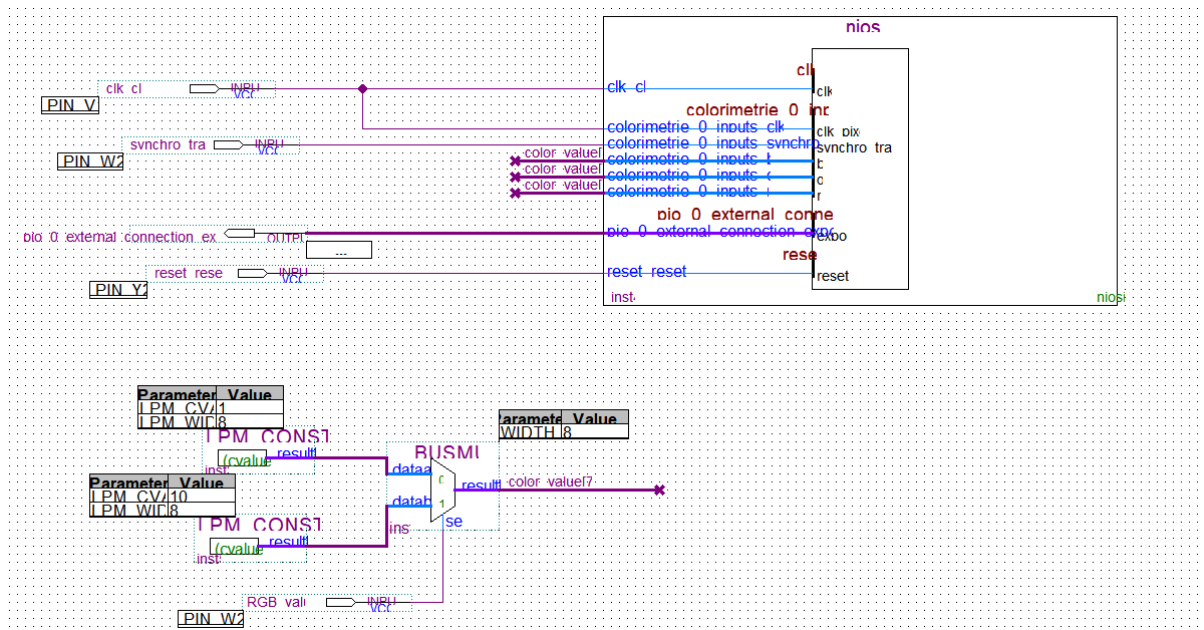
Une fois qu'on a vérifié le bon fonctionnement de l'IP, on l'enveloppe l'IP colorimetrie pour qu'il puisse être inclus au SoPC et relié au Nios II via un bus Avalon Memory Mapped.



Il est ensuite ajouté à notre SoPC à partir du .qsys.

	colorimetrie_0 avalon_slave_0 reset clock inputs	colorimetrie Avalon Memory Mapped ... Reset Input Clock Input Conduit	Double-click to Double-click to Double-click to colorimetrie_0 ...	[clock] [clock] clk_0 [clock]	# 0x0004 1030 0x0004_103f
---	---	---	---	--	------------------------------

Un fois le code du SoPC regénérer, avec cet nouvel IP, on connecte les nouvelles entrées, comme ci-dessous :



Comme on essaye surtout de tester l'IP :

- Les entrées RGB sont connectées à une valeur constante, grâce à un multiplexeur, soit 1, soit 10, en fonction de l'état d'un des switch de la carte.
- L'entrée clk_pixel est elle connectée à la clock de l'ensemble du système.
- L'entrée synchro_frame est lui connecté à un switch de la carte.

On écrit ensuite pour le nios, le programme c suivant :

```
int main()
{
    printf("Hello from Nios II!\n");
    char r_value;
    char g_value;
    char b_value;

    while(1)
    {
        IOWR(COLORIMETRIE_0_BASE, 0, 0);

        // On lit mtn les valeurs
        r_value = IORD(COLORIMETRIE_0_BASE, 0x1);
        g_value = IORD(COLORIMETRIE_0_BASE, 0x2);
        b_value = IORD(COLORIMETRIE_0_BASE, 0x3);

        printf("valeur rgb %d %d %d \n", r_value, g_value, b_value);

        usleep(500000);
    }
    return 0;
}
```

Il permet toute les demi secondes de :

- D'écrire sur le bus Avalon (peu importe l'adresse et la donnée écrite) et ainsi activer les signaux chipselect et write de l'IP, ce qui permet une écriture dans les registres des valeurs d'entrées rgb.
- De lire dans les registres R (à l'adresse 0x1), G (à l'adresse 0x2), G (à l'adresse 0x2), leurs moyennes respectives.

On obtient le résultat suivant :

- le switch contrôlant synchro_frame qui permet bien la remise à niveau des compteurs de moyenne
- en mettant la valeur 10 sur deux cycles, on a bien une moyenne de 10
- puis en ajoutant la valeur 1 sur un cycle, on a bien une moyenne de $(10 + 10 + 1) = 7$
- et ainsi de suite.

```
valeur rgb 0 0 0
valeur rgb 10 10 10
valeur rgb 10 10 10
valeur rgb 7 7 7
valeur rgb 5 5 5
valeur rgb 4 4 4
valeur rgb 4 4 4
valeur rgb 3 3 3
```

Partie 3 : Affichage graphique de l'indice colorimétrique

I. Conception d'un IP VGA

Comme je travaille sur une carte DE10 Nano qui à un port HDMI et non VGA, je vais concevoir dans cette partie, un contrôleur hdmi.

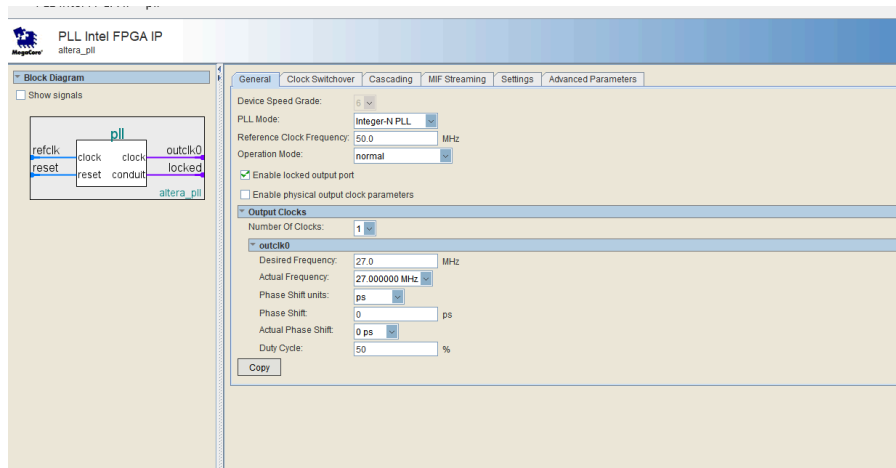
Q1. Le composant qui va permettre de mesurer le temps est le compteur.

Q2. Le VGA et HDMI transmettent les pixels ligne par ligne, puis image par image. Les signaux de synchronisation HS et VS, permettent de respectivement indiquer une fin de ligne et une fin de frame.

Ils peuvent ainsi être générés en utilisant des compteurs synchronisés sur l'horloge système.

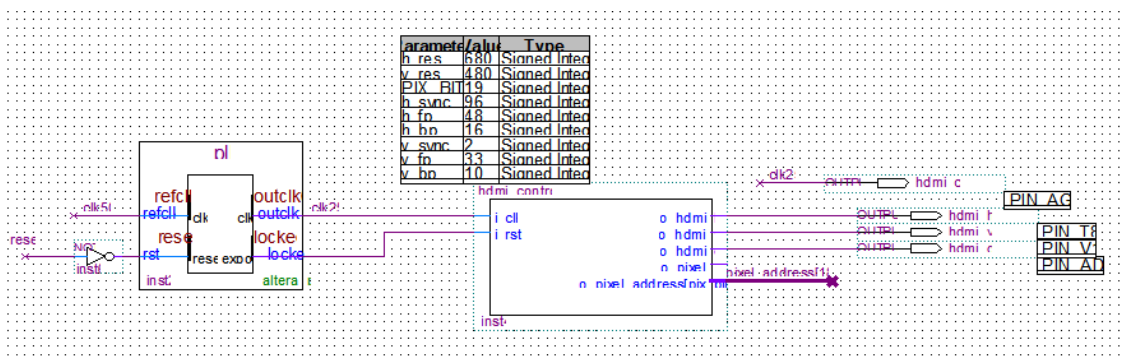
Q3. Diviseur de clock 50 MHz -> 25 MHz

Pour obtenir un diviseur de clock, j'utilise l'IP ALTERA_PLL disponible dans l'IP catalog que je configure comme ci-dessous.



Q4. Pour le contrôleur HDMI, on fait le programme [hdmi_controler.vhdl](#) en annexe, au lieu de faire des schéma bloc par préférence personnelle.

On connecte la pll au contrôleur hdmi comme suivant et on y ajoute les signaux de sorties qu'on assigne grâce au pin planner et la documentation.



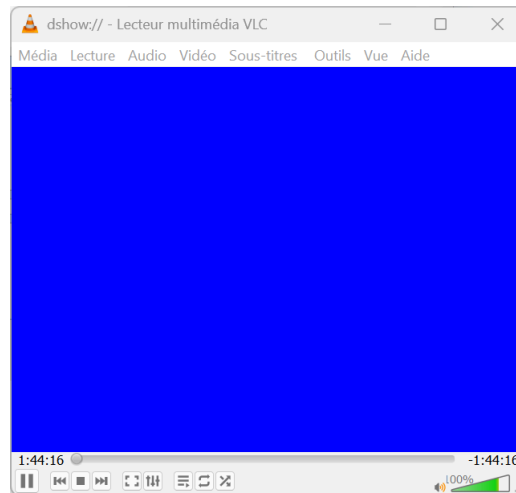
Out	Signal	Type	Pin	Width	Bus	Pin	Width	Bus	Pin	Width	Bus
out	hdmi_clk	Output	PIN_AG5	3B	B3B_N0	PIN_AG5	3.3-V	LVTTTL			
out	hdmi_data[23]	Output	PIN_AE8	3B	B3B_N0	PIN_AE8	3.3-V	LVTTTL			
out	hdmi_data[22]	Output	PIN_AF9	3B	B3B_N0	PIN_AF9	3.3-V	LVTTTL			
out	hdmi_data[21]	Output	PIN_AG6	3B	B3B_N0	PIN_AG6	3.3-V	LVTTTL			
out	hdmi_data[20]	Output	PIN_AH6	3B	B3B_N0	PIN_AH6	3.3-V	LVTTTL			
out	hdmi_data[19]	Output	PIN_AH5	3B	B3B_N0	PIN_AH5	3.3-V	LVTTTL			
out	hdmi_data[18]	Output	PIN_AH4	3B	B3B_N0	PIN_AH4	3.3-V	LVTTTL			
out	hdmi_data[17]	Output	PIN_AH2	3B	B3B_N0	PIN_AH2	3.3-V	LVTTTL			
out	hdmi_data[16]	Output	PIN_AE4	3B	B3B_N0	PIN_AE4	3.3-V	LVTTTL			
out	hdmi_data[15]	Output	PIN_AF5	3B	B3B_N0	PIN_AF5	3.3-V	LVTTTL			
out	hdmi_data[14]	Output	PIN_AF8	3B	B3B_N0	PIN_AF8	3.3-V	LVTTTL			
out	hdmi_data[13]	Output	PIN_AF6	3B	B3B_N0	PIN_AF6	3.3-V	LVTTTL			
out	hdmi_data[12]	Output	PIN_AE7	3B	B3B_N0	PIN_AE7	3.3-V	LVTTTL			
out	hdmi_data[11]	Output	PIN_AB4	3A	B3A_N0	PIN_AB4	3.3-V	LVTTTL			
out	hdmi_data[10]	Output	PIN_AE9	3B	B3B_N0	PIN_AE9	3.3-V	LVTTTL			
out	hdmi_data[9]	Output	PIN_Y4	3A	B3A_N0	PIN_Y4	3.3-V	LVTTTL			
out	hdmi_data[8]	Output	PIN_AF10	3B	B3B_N0	PIN_AF10	3.3-V	LVTTTL			
out	hdmi_data[7]	Output	PIN_Y5	3A	B3A_N0	PIN_Y5	3.3-V	LVTTTL			
out	hdmi_data[6]	Output	PIN_AE11	3B	B3B_N0	PIN_AE11	3.3-V	LVTTTL			
out	hdmi_data[5]	Output	PIN_AD10	3B	B3B_N0	PIN_AD10	3.3-V	LVTTTL			
out	hdmi_data[4]	Output	PIN_AD11	3B	B3B_N0	PIN_AD11	3.3-V	LVTTTL			
out	hdmi_data[3]	Output	PIN_Y8	3A	B3A_N0	PIN_Y8	3.3-V	LVTTTL			
out	hdmi_data[2]	Output	PIN_W8	3A	B3A_N0	PIN_W8	3.3-V	LVTTTL			
out	hdmi_data[1]	Output	PIN_AE12	3B	B3B_N0	PIN_AE12	3.3-V	LVTTTL			
out	hdmi_data[0]	Output	PIN_AD12	3B	B3B_N0	PIN_AD12	3.3-V	LVTTTL			
out	hdmi_de	Output	PIN_AD19	4A	B4A_N0	PIN_AD19	3.3-V	LVTTTL			
out	hdmi_hs	Output	PIN_T8	3A	B3A_N0	PIN_T8	3.3-V	LVTTTL			
in	hdmi_int	Input	PIN_AF11	3B	B3B_N0	PIN_AF11	3.3-V	LVTTTL			
out	hdmi_vs	Output	PIN_V13	4A	B4A_N0	PIN_V13	3.3-V	LVTTTL			

En mettant :

- R (hdmi_data[23..16]) = 0

- $G(\text{hmdi_data}[15..8]) = 0$
- $B(\text{hmdi_data}[7..0]) = 255$

On a le résultat suivant :



II. Gestion par le Nios II

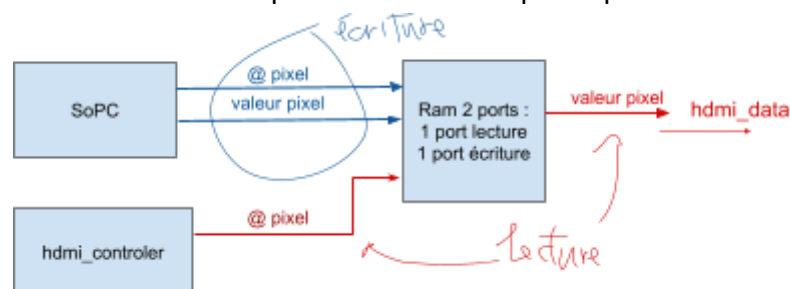
Q9. D'après la datasheet, il y a "5,570 Kbits embedded memory".

Sachant que le système avec le SoPC contient un mémoire on-chip memory de 128 Ko, soit 1024 Kbits, il nous reste 4546 Kbits pour mettre en place une mémoire pour l'affichage de la courbe.

Q10.

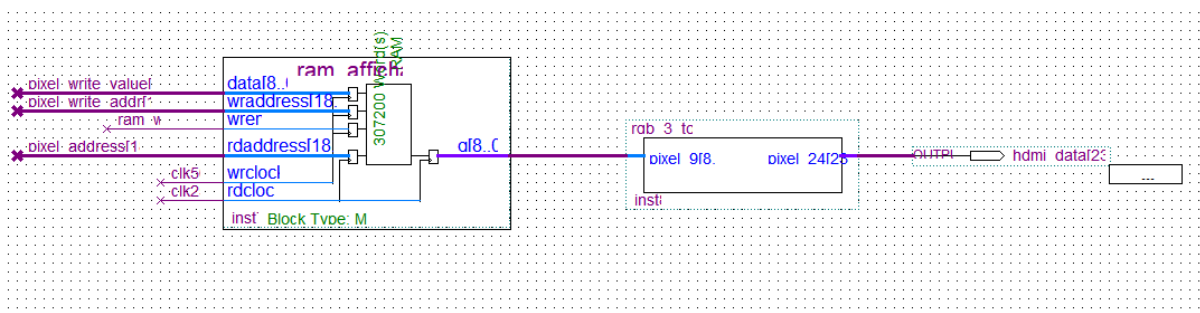
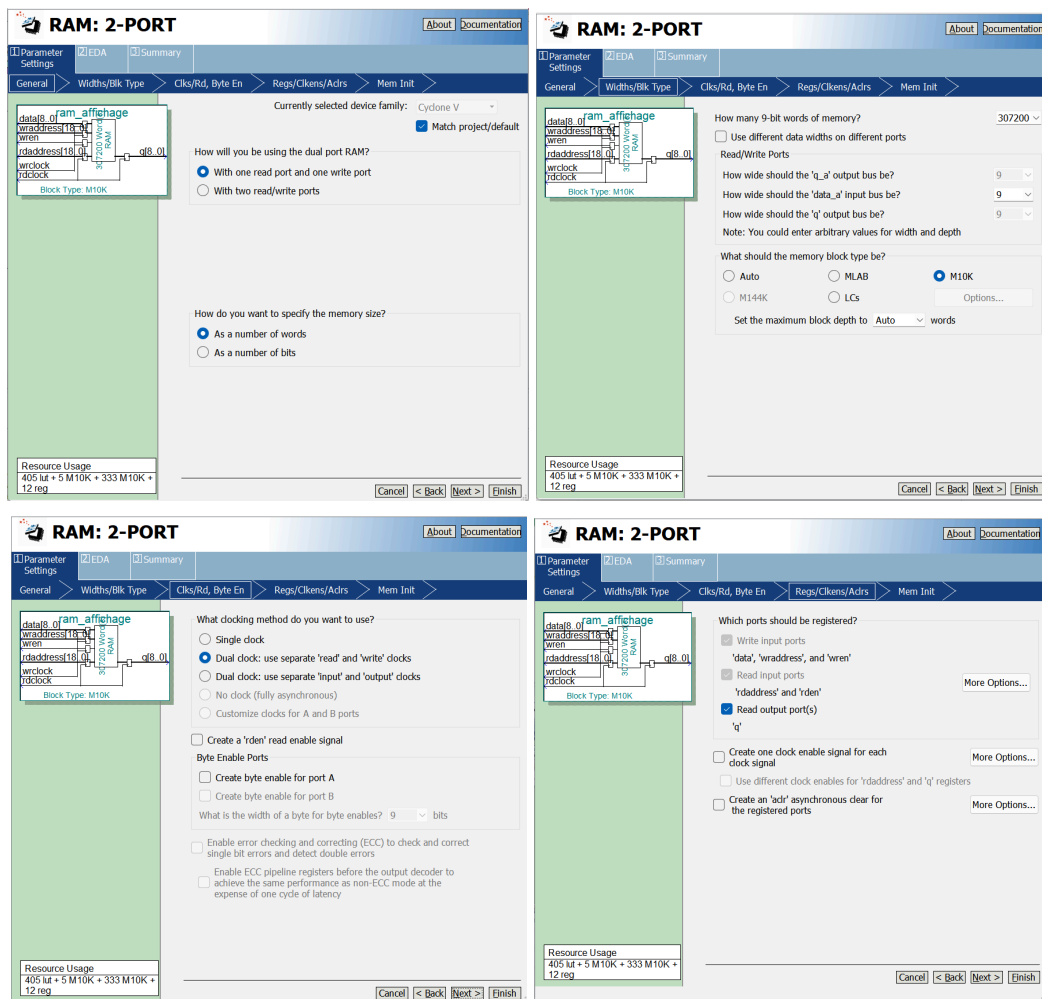
On propose d'utiliser un mémoire double port pour la mémorisation des pixels d'affichage du hdmi avec :

- 1 port utilisé exclusivement pour l'écriture des pixels par le SoPC
- 1 port utilisé exclusivement pour la lecture des pixels par le hdmi



Comme `hdmi_data` est sur 24 bits, avec 8 bits pour chaque composante couleur, il aurait fallu une mémoire avec des mots de 24 bits, avec 640x480 adresses, correspondant à 7,372,800 bits. Néanmoins cette quantité de mémoire n'est pas disponible, on décide donc d'utiliser des mots de 9 bits, soit 3 bits pour chaque composante couleur, soit 2,764,800 bits.

On ajoute à partir de l'IP catalog, une ram double port, avec un port pour l'écriture et un port pour la lecture qu'on configuré comme ci-dessous :



- L'horloge d'écriture wrclck est cadencé par l'horloge système à 50 MHz pareil que l'IP colorimetrie qui écrit dans la RAM pour afficher la courbe
- L'adresse du pixel écrit et sa valeur est générée la l'IP colorimetrie
- L'horloge de lecture rdclck est cadencé par l'horloge pixel du hdmi à 25 MHz.
- L'adresse du pixel lit est généré par le hdmi_controler

- Si le switch SW2 à l'état logique 0
 - R = 10, G = 1, B = 10
 - $\Rightarrow R/G = 10$
- Si le switch SW2 à l'état logique 1
 - R = 10, G = 10, B = 10
 - $\Rightarrow R/G = 1$

On peut ensuite finir de programmer le Nios avec [Programme Nios](#) pour afficher une courbe colorimétrique.
On obtient le résultat suivant :

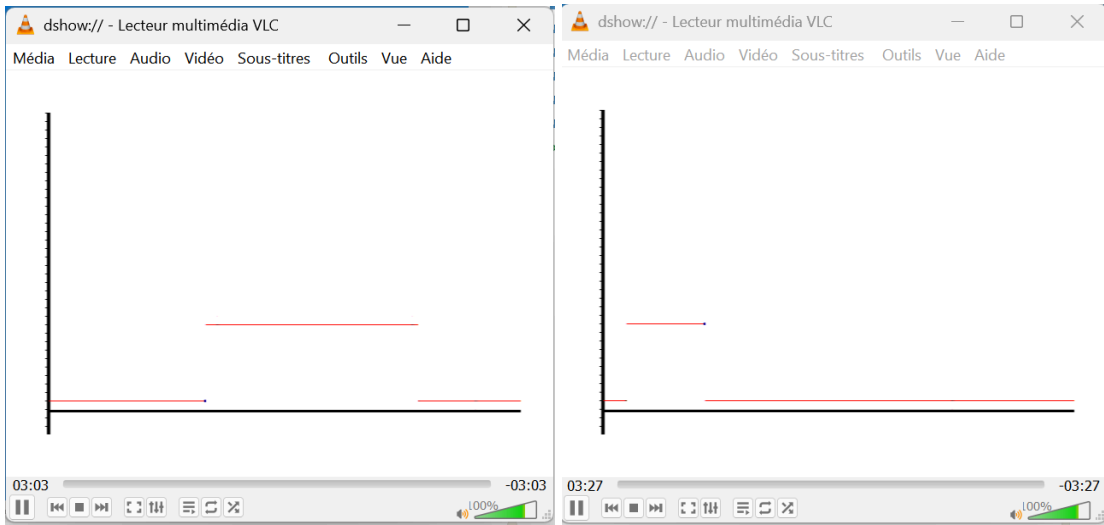
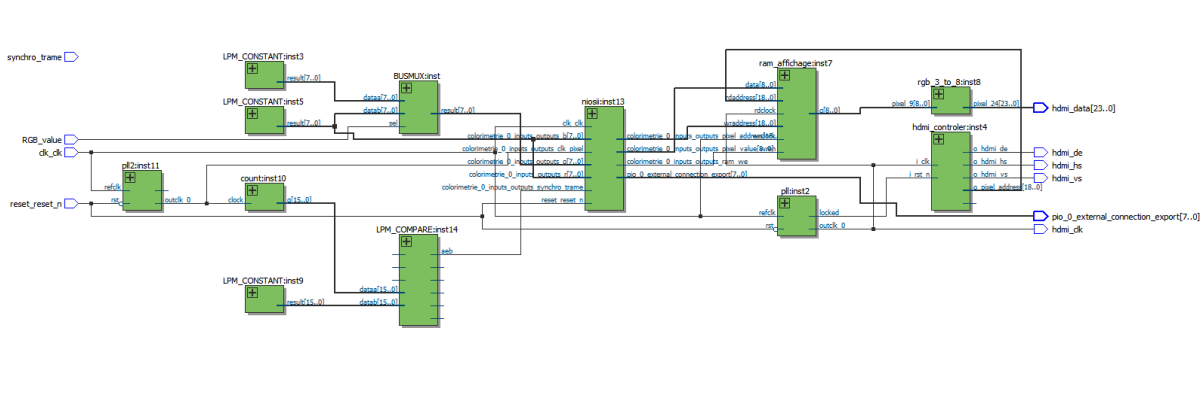


Schéma RTL finale :



Analysis & Synthesis Summary	
Analysis & Synthesis Status	Successful - Fri Feb 13 17:55:13 2026
Quartus II 64-Bit Version	15.0.0 Build 145 04/22/2015 SJ Web Edition
Revision Name	videocapsule
Top-level Entity Name	videocapsule
Family	Cyclone V
Logic utilization (in ALMs)	N/A
Total registers	2372
Total pins	40
Total virtual pins	0
Total block memory bits	3,859,840
Total DSP Blocks	3
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	2
Total DLLs	0

Annexe :

colorimetrie.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity colorimetrie is
    port (
        address : in std_logic_vector(1 downto 0);
        writedata : in std_logic_vector(31 downto 0);
        R : in std_logic_vector(7 downto 0);
        G : in std_logic_vector(7 downto 0);
        B : in std_logic_vector(7 downto 0);
        clk_pixel : in std_logic;
        synchro_trame : in std_logic;

        cs : in std_logic;
        wr : in std_logic;
        rd : in std_logic;
        reset : in std_logic;
        clk : in std_logic;

        readdata : out std_logic_vector(31 downto 0)
    );
end colorimetrie;

architecture arch of colorimetrie is
    constant SUM_MAX : positive := 22848000;
    constant PX_MAX : positive := 89600;      --320*280

    signal sum_r : natural range 0 to SUM_MAX := 0;
    signal sum_g : natural range 0 to SUM_MAX := 0;
    signal sum_b : natural range 0 to SUM_MAX := 0;

    signal nb_value_r : natural range 0 to PX_MAX := 0;
    signal nb_value_g : natural range 0 to PX_MAX := 0;
    signal nb_value_b : natural range 0 to PX_MAX := 0;

begin
    process(clk, reset)
    begin
        if (reset = '1') then
            readdata <= (others => '0');

        elsif rising_edge(clk) then

            if (cs = '1' and rd = '1') then

                if (address = "01") then
                    if nb_value_r /= 0 then
                        readdata <=
std_logic_vector(to_unsigned(sum_r/nb_value_r, 32));
                    else
                        readdata <= std_logic_vector(to_unsigned(sum_r,
32));
```

```

        end if;
        elsif (address = "10") then
            if nb_value_g /= 0 then
                readdata <=
std_logic_vector(to_unsigned(sum_g/nb_value_g, 32));
            else
                readdata <= std_logic_vector(to_unsigned(sum_g,
32));
            end if;
        elsif (address = "11") then
            if nb_value_b /= 0 then
                readdata <=
std_logic_vector(to_unsigned(sum_b/nb_value_b, 32));
            else
                readdata <= std_logic_vector(to_unsigned(sum_b,
32));
            end if;
        end if;
    end if;

end process;

process (clk_pixel, reset)
begin

    if (reset = '1') then
        sum_r <= 0;
        sum_g <= 0;
        sum_b <= 0;
        nb_value_r <= 0;
        nb_value_g <= 0;
        nb_value_b <= 0;

    elsif rising_edge(clk_pixel) then
        if (cs = '1' and wr = '1') then

            if (synchro_trame = '1') then
                sum_r <= 0;
                sum_g <= 0;
                sum_b <= 0;
                nb_value_r <= 0;
                nb_value_g <= 0;
                nb_value_b <= 0;
            else
                sum_r <= sum_r + to_integer(unsigned(R));
                sum_g <= sum_g + to_integer(unsigned(G));
                sum_b <= sum_b + to_integer(unsigned(B));
                nb_value_r <= nb_value_r + 1;
                nb_value_g <= nb_value_g + 1;
                nb_value_b <= nb_value_b + 1;
            end if;
        end if;
    end if;
end process;

end architecture arch;

```

hdmi_controler.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hdmi_controler is
    generic (
        -- Resolution
        h_res    : positive := 680;
        v_res    : positive := 480;

        PIX_BITS : positive := 19;

        -- Timings
        h_sync    : positive := 96;
        h_fp      : positive := 48;
        h_bp      : positive := 16;

        v_sync    : positive := 2;
        v_fp      : positive := 33;
        v_bp      : positive := 10
    );
    port (
        i_clk      : in std_logic;
        i_rst_n    : in std_logic;

        o_hdmi_hs  : out std_logic;
        o_hdmi_vs  : out std_logic;
        o_hdmi_de  : out std_logic;

        o_pixel_en : out std_logic;

        o_pixel_address : out unsigned(PIX_BITS-1 downto 0)
    );
end hdmi_controler;

architecture rtl of hdmi_controler is
    constant h_start: positive := h_sync + h_fp;
    constant h_end  : positive := h_res + h_start;
    constant h_total: positive := h_end + h_bp;

    constant v_start: positive := v_sync + v_fp;
    constant v_end  : positive := v_res + v_start;
    constant v_total: positive := v_end + v_bp;

    constant pixel_number : natural := h_res*v_res;

    signal r_h_count : natural range 0 to h_total := 0;
    signal r_h_active: std_logic := '0';

    signal r_v_count : natural range 0 to v_total := 0;
    signal r_v_active: std_logic := '0';

    signal s_x_counter : natural range 0 to (h_res - 1) := 0;
    signal s_y_counter : natural range 0 to (v_res - 1) := 0;
begin
    -- Horizontal control signals
    process(i_clk, i_rst_n)
    begin
        if (i_rst_n = '0') then
            r_h_count <= 0;
            o_hdmi_hs <= '1';
            r_h_active <= '0';
        elsif rising_edge(i_clk) then
            if (r_h_count = h_total) then
                r_h_count <= 0;
            else
                r_h_count <= r_h_count + 1;
            end if;
        end if;
    end process;
end rtl;
```

```

        if ((r_h_count >= h_sync) and (r_h_count /= h_total)) then
            o_hdmi_hs <= '1';
        else
            o_hdmi_hs <= '0';
        end if;

        if (r_h_count = h_start) then
            r_h_active <= '1';
        elsif (r_h_count = h_end) then
            r_h_active <= '0';
        end if;
    end if;
end process;

-- Vertical control signals
process(i_clk, i_rst_n)
begin
    if (i_rst_n = '0') then
        r_v_count <= 0;
        o_hdmi_vs <= '1';
        r_v_active <= '0';
    elsif rising_edge(i_clk) then
        if (r_h_count = h_total) then
            if (r_v_count = v_total) then
                r_v_count <= 0;
            else
                r_v_count <= r_v_count + 1;
            end if;

            if ((r_v_count >= v_sync) and (r_v_count /= v_total)) then
                o_hdmi_vs <= '1';
            else
                o_hdmi_vs <= '0';
            end if;

            if (r_v_count = v_start) then
                r_v_active <= '1';
            elsif (r_v_count = v_end) then
                r_v_active <= '0';
            end if;
        end if;
    end if;
end process;

-- Display enable and dummy pixels
process(i_clk, i_rst_n)
begin
    if (i_rst_n = '0') then
        o_hdmi_de <= '0';
    elsif rising_edge(i_clk) then
        o_hdmi_de <= r_v_active and r_h_active;
    end if;
end process;

-- Generate address
o_pixel_en <= '1' when (r_v_active = '1') and (r_h_active = '1') else '0';

s_x_counter <= (r_h_count - h_start - 1) when (r_h_count > h_start) and (r_h_count < h_res + h_start + 1)
else 0;
s_y_counter <= (r_v_count - v_start - 1) when (r_v_count > v_start) and (r_v_count < v_res + v_start + 1)
else 0;

o_pixel_address <= to_unsigned(s_x_counter + (s_y_counter * h_res), PIX_BITS); --r_pixel_counter;

end architecture rtl;

```

rgb_3_to_8.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity rgb_3_to_8 is
    port (
        pixel_9  : in  std_logic_vector(8 downto 0); -- RRR GGG BBB
        pixel_24 : out std_logic_vector(23 downto 0) -- RRRRRRRR GGGGGGGG BBBBBBBB
    );
end entity;

architecture rtl of rgb_3_to_8 is
    signal R3 : std_logic_vector(2 downto 0);
    signal G3 : std_logic_vector(2 downto 0);
    signal B3 : std_logic_vector(2 downto 0);

begin

    -- Extraction
    R3 <= pixel_9(8 downto 6);
    G3 <= pixel_9(5 downto 3);
    B3 <= pixel_9(2 downto 0);

    -- Expansion 3 bits -> 8 bits (bit replication)
    pixel_24(23 downto 16) <= R3 & R3 & R3(2 downto 1);
    pixel_24(15 downto 8)  <= G3 & G3 & G3(2 downto 1);
    pixel_24(7 downto 0)   <= B3 & B3 & B3(2 downto 1);

end architecture;
```

colorimetrie.vhd 2

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity colorimetrie is
port (
    address : in std_logic_vector(2 downto 0);
    writedata : in std_logic_vector(31 downto 0);
    R : in std_logic_vector(7 downto 0);
    G : in std_logic_vector(7 downto 0);
    B : in std_logic_vector(7 downto 0);
    clk_pixel : in std_logic;
    synchro_trame : in std_logic;

    cs : in std_logic;
    wr : in std_logic;
    rd : in std_logic;
    reset : in std_logic;
    clk : in std_logic;

    readdata : out std_logic_vector(31 downto 0);

    pixel_address : out std_logic_vector(18 downto 0);
    pixel_value : out std_logic_vector(8 downto 0);
    ram_we : out std_logic
);
end colorimetrie;

architecture arch of colorimetrie is
    constant SUM_MAX : positive := 22848000;
    constant PX_MAX : positive := 89600; --320*280

    signal sum_r : natural range 0 to SUM_MAX := 0;
    signal sum_g : natural range 0 to SUM_MAX := 0;
    signal sum_b : natural range 0 to SUM_MAX := 0;

    signal nb_value_r : natural range 0 to PX_MAX := 0;
    signal nb_value_g : natural range 0 to PX_MAX := 0;
    signal nb_value_b : natural range 0 to PX_MAX := 0;

begin
    process(clk, reset)
    begin
        if (reset = '1') then
            readdata <= (others => '0');

        elsif rising_edge(clk) then

            if (cs = '1' and rd = '1') then

                if (address = "001") then
                    if nb_value_r /= 0 then
                        readdata <= std_logic_vector(to_unsigned(sum_r/nb_value_r, 32));
                    else
                        readdata <= std_logic_vector(to_unsigned(sum_r, 32));
                    end if;
                elsif (address = "010") then
                    if nb_value_g /= 0 then
                        readdata <= std_logic_vector(to_unsigned(sum_g/nb_value_g, 32));
                    else
                        readdata <= std_logic_vector(to_unsigned(sum_g, 32));
                    end if;
                elsif (address = "011") then
                    if nb_value_b /= 0 then
                        readdata <= std_logic_vector(to_unsigned(sum_b/nb_value_b, 32));
                    else
```

```

        readdata <= std_logic_vector(to_unsigned(sum_b, 32));
    end if;
end if;

elsif (cs = '1' and wr = '1') then

    -- ECRITURE ADDRESS 101 => écriture pixel_address
    if (address = "101") then
        pixel_address <= writedata(18 downto 0);

    -- ECRITURE ADDRESS 110 => écriture pixel_value
    elsif (address = "110") then
        pixel_value <= writedata(8 downto 0);

    -- ECRITURE ADDRESS 111 => écriture ram_we
    elsif (address = "111") then
        ram_we <= writedata(0);

    end if;

end if;

end if;

end process;

process (clk_pixel, reset)
begin

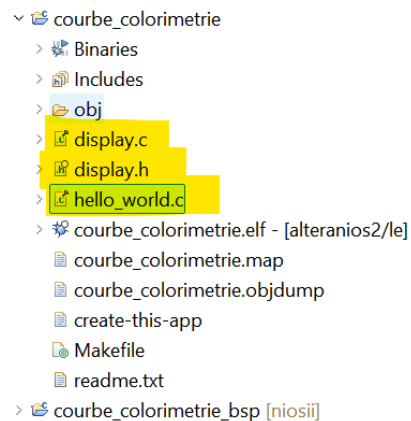
    if (reset = '1') then
        sum_r <= 0;
        sum_g <= 0;
        sum_b <= 0;
        nb_value_r <= 0;
        nb_value_g <= 0;
        nb_value_b <= 0;

    elsif rising_edge(clk_pixel) then
        if (synchro_trame = '1') then
            sum_r <= 0;
            sum_g <= 0;
            sum_b <= 0;
            nb_value_r <= 0;
            nb_value_g <= 0;
            nb_value_b <= 0;
        else
            sum_r <= sum_r + to_integer(unsigned(R));
            sum_g <= sum_g + to_integer(unsigned(G));
            sum_b <= sum_b + to_integer(unsigned(B));
            nb_value_r <= nb_value_r + 1;
            nb_value_g <= nb_value_g + 1;
            nb_value_b <= nb_value_b + 1;
        end if;
    end if;
end process;

end architecture arch;

```

Programme Nios



display.h

```
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <io.h>

#include "system.h"
#include "display.h"

#define READ_R 0x1
#define READ_G 0x2
#define READ_B 0x3

#define WRITE_RGB 0x4
#define WRITE_PIXEL_ADDR 0x5
#define WRITE_PIXEL_VALUE 0x6
#define WRITE_RAM_WE 0x7

#define H_RES 640
#define V_RES 480
#define PIXEL_ADDR_MAX H_RES*V_RES

#define PIXEL(x, y) ((y) * H_RES + (x))

#define DEBUT_AXE_X 50
#define FIN_AXE_X 600
#define ORIGIN_AXE_Y 400

#define COLOR(r, g, b) (r << 6) | (g << 3) | b
#define WHITE COLOR(7, 7, 7)
#define BLACK COLOR(0, 0, 0)
#define RED COLOR(7, 0, 0)
#define GREEN COLOR(0, 7, 0)
#define BLUE COLOR(0, 0, 7)

void display_pixel(int pos_x, int pos_y, short color);
void display_clean();
void display_repere();
```

display.c

```
void display_pixel(int pos_x, int pos_y, short color)
{
    if (pos_x < H_RES && pos_y < V_RES)
    {
        IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x0);
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_VALUE, color);
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(pos_x, pos_y));
        IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x1);
    }
}

void display_clean()
{
    int i;
    IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_VALUE, WHITE);
    IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x1);
    for (i = 0; i < PIXEL_ADDR_MAX; i++)
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, i);
    IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x0);
}

void display_repere()
{
    int i;
    IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_VALUE, BLACK);
    IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x1);
    for (i = 50; i < 430; i++)
    {
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(DEBUT_AXE_X-1, i));
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(DEBUT_AXE_X-2, i));
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(DEBUT_AXE_X-3, i));
    }
    for (i = 50; i < 600; i++)
    {
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(i, ORIGIN_AXE_Y+1));
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(i, ORIGIN_AXE_Y+2));
        IOWR(COLORIMETRIE_0_BASE, WRITE_PIXEL_ADDR, PIXEL(i, ORIGIN_AXE_Y+3));
    }
    IOWR(COLORIMETRIE_0_BASE, WRITE_RAM_WE, 0x0);

    for (i = 50; i < 430; i+=10)
    {
        display_pixel(DEBUT_AXE_X-4, i, BLACK);
        display_pixel(DEBUT_AXE_X-5, i, BLACK);
        display_pixel(DEBUT_AXE_X-6, i, BLACK);
    }
}
```

hello_world.c

```
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <io.h>
```

```

#include "system.h"
#include "display.h"

int main()
{
    printf("Hello from Nios II!\n");

    uint8_t r_value;
    uint8_t g_value;
    int indice_rg;
    int courbe_rg[FIN_AXE_X];
    int courbe_index = DEBUT_AXE_X;

    display_clean();
    display_repere();

    while(1)
    {
        r_value = IORD(COLORIMETRIE_0_BASE, 0x1);
        g_value = IORD(COLORIMETRIE_0_BASE, 0x2);

        // printf("valeur rgb %d %d %d \n", r_value, g_value, b_value);

        if (g_value != 0)
        {
            indice_rg = r_value / g_value;
            // printf("indice rg : %d\n", indice_rg);

            // on efface l'ancienne valeur
            display_pixel(courbe_index, courbe_rg[courbe_index], WHITE);

            // on nettoie la valeur précédente
            if (courbe_index > DEBUT_AXE_X)
            {
                display_pixel(courbe_index-1, courbe_rg[courbe_index-1], RED);
                display_pixel(courbe_index-1, courbe_rg[courbe_index-1]+1, WHITE);
                display_pixel(courbe_index-1, courbe_rg[courbe_index-1]-1, WHITE);
            }
            else if (courbe_index == DEBUT_AXE_X)
            {
                display_pixel(FIN_AXE_X, courbe_rg[FIN_AXE_X], RED);
                display_pixel(FIN_AXE_X, courbe_rg[FIN_AXE_X]+1, WHITE);
                display_pixel(FIN_AXE_X, courbe_rg[FIN_AXE_X]-1, WHITE);
            }

            // on garde la nouvelle valeur et l'affiche le nouveau point en bleu
            courbe_rg[courbe_index] = ORIGIN_AXE_Y - 10*indice_rg;
            display_pixel(courbe_index, courbe_rg[courbe_index], BLUE);
            display_pixel(courbe_index, courbe_rg[courbe_index]+1, BLUE);
            display_pixel(courbe_index, courbe_rg[courbe_index]-1, BLUE);

            if (courbe_index >= FIN_AXE_X)
                courbe_index = DEBUT_AXE_X;
            else
                courbe_index++;
        }
        usleep(50000);
    }
    return 0;
}

```